



TP1 : Manipulation des tableaux/matrices avec numpy

1. Outils de machine learning avec Python

Pour faire du machine learning et de l'analyse de données avec Python, on a principalement besoin des bibliothèques Python suivantes :

- **Numpy** : fournit les structures de données fondamentales utilisées par scikit-learn particulièrement les tableaux multidimensionnels. Les données d'entrée pour scikit-learn sont typiquement des tableaux numpy.
- **Pandas** : fournit des structures de données comme DataFrame qui peut être vue comme un spreadsheet table avec des lignes et des colonnes titrées. Contrairement aux tableaux de Numpy, les colonnes dans un DataFrame peuvent être de types différents.
- **Matplotlib** : permet de tracer et visualiser des données sous formes de graphiques. Ses modules pyplot, seaborn et graphviz sont les plus utilisés.
- **Scikit-learn** : une bibliothèque Python destinée à l'apprentissage automatique.

2. Présentation du package Numpy

Numpy est un package pour Python spécialisé dans la manipulation des tableaux (array), pour nous essentiellement les vecteurs et les matrices. Le package propose un grand nombre de routines pour un accès rapide aux données (ex. recherche, extraction), pour les manipulations diverses (ex. tri), pour les calculs (ex. calcul statistique).

Les tableaux « numpy » sont plus performants (rapidité, gestion de la volumétrie) que les collections usuelles de Python. Ils sont sous-jacents à de nombreux packages dédiés au calcul scientifique sous Python.

Travail à faire :

Ouvrez Jupyter Notebook.

3. Vecteurs et matrices

Une matrice est un tableau (array) à 2 dimensions.

Travail à faire :

1. Importez la bibliothèque numpy : **import numpy as np**
2. Créez le vecteur ligne suivant avec numpy : $L1 = (10, 20, 30, 40)$
3. Créez un vecteur colonne C1 contenant les mêmes valeurs que L1 :

$$C1 = \begin{pmatrix} 10 \\ 20 \\ 30 \\ 40 \end{pmatrix}$$

4. Comment obtenir C1 à partir de L1 ?
5. Créez la matrice M1 suivante avec numpy :

1	2	3
4	5	6
7	8	9

6. Affichez les éléments de la 1^{ère} colonne de M1
7. Affichez les éléments de la 2^{ème} ligne de M1
8. Affichez seulement les deux dernières colonnes de M1
9. Affichez la taille de la matrice et le nombre d'éléments de M1
10. Soit le vecteur suivant : $a = \text{np.array}([2.1, 3.4, 6.7, 8.1, 3.5, 7.2])$, convertissez-le en une matrice A de 3 lignes et 2 colonnes.
11. Affichez le type des éléments de A
12. Affichez le transposé de A
13. Calculez le produit matriciel $A * A^t$

3. Calculs statistiques sur les matrices

Les calculs sont réalisés selon une certaine organisation des données (axis = None : toutes les valeurs prises globalement ; axis = 0 : traitement par colonne ; axis = 1 : traitement par ligne).

Par exemple, pour afficher la moyenne par colonne, on écrit : `print(np.mean(v,axis=0))`

Travail à faire :

1. Soit le vecteur T suivant :

$$\begin{pmatrix} 1 \\ 3 \\ -2 \\ 5 \end{pmatrix}$$

- a) Créez ce vecteur avec numpy
- b) Triez le dans l'ordre croissant (par défaut)
- c) Affichez son minimum

2. Soit la matrice M suivante :

$$\begin{matrix} 5 & 0 & 0 \\ 3 & 7 & 9 \\ 1 & 8 & -1 \end{matrix}$$

- a) Calculez la somme de tous les éléments de la matrice M
- b) Calculez la somme de chaque colonne de la matrice M
- c) Calculez la somme de chaque ligne de la matrice M
- d) Calculez le minimum et maximum de tous les éléments de la matrice M
- e) Calculez le minimum et maximum de chaque colonne de la matrice M
- f) Retournez la position du minimum de chaque ligne de la matrice M
- g) Remplacez tous les éléments de M qui sont inférieurs à 5 par le minimum de M